

数据架构演进

从小蚕 StarRocks 存算一体到 Paimon + StarRocks 存算分离

现状：StarRocks 被当成全能层

StarRocks 同时承接 OLAP (分析) + OLTP (点查) + 存储，后端服务直接查 StarRocks 时延迟不可控。

当前问题架构

业务数据源 → StarRocks 存算一体 (单集群) → 看板 / 报表 / 后端服务
后端 OLTP 点查也走 StarRocks → 高峰期 3 秒超时，I/O 不可隔离

存算一体在腾讯云上的隔离困境

Workload Group 只能做软隔离

CPU 和内存可以按 query 分组划分，但磁盘 I/O 和网络带宽是所有 Workload Group 共享的。后端点查扫热数据时吃 I/O，分析侧大查询 GROUP BY 全表扫描也吃 I/O —— 两股流量在同一个 BE 进程里抢同一块盘，谁也躲不开。

扩容绑死

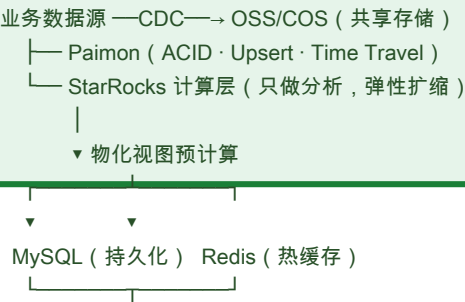
想给分析侧加算力就必须同时加存储 (每扩一个 BE 都带一块盘)，想扩存储容量也得捎上计算。成本不是线性增长的，是按节点整体翻倍的。高峰期分析查询和点查并发上去，只能整体升配，没法单独给某一层加弹性。

实践验证

此前试图用「再搭一套 StarRocks 集群给后端专用」来缓解，结果是：存储数据要写两份，两套集群各自维护，单集群内部的 I/O 竞争依然存在，本质上是用冗余成本换一个不彻底的解法。

目标架构：Paimon + StarRocks 存算分离 + MySQL/Redis

目标架构 — 各层物理隔离



各层职责

层	角色
Paimon	湖格式，管实时写入、Upsert、ACID、Time Travel
StarRocks	只做分析查询，挂载 Paimon 外表，存算分离弹性扩缩
MySQL	OLTP 持久化层，后端点查的最终出口
Redis	热缓存层，高频指标预计算后缓存
后端服务	只访问 MySQL/Redis，延迟可控

核心变化对比

现状	改造后
StarRocks 扛写入	Paimon 扛写入
StarRocks 扛点查	MySQL/Redis 扛点查
一套集群不分家	分析和服分层
锁在存算一体，扩缩绑死	存算分离，计算弹性扩缩，存储按量付费
Workload Group 软隔离，I/O 不可控	各层物理隔离，I/O 路径完全分开
后端 3 秒超时	后端 50ms 以内
两套集群双写，冗余成本高	统一存储底座，无数据冗余

成本分析

当前成本结构

存算一体下 BE 节点捆绑 CPU + 内存 + SSD，成本以节点为单位整体翻。此前两套集群双写方案，存储和算力各翻一倍，用冗余换隔离，边际成本线性甚至超线性增长。

目标架构成本拆解

组件	角色	成本特征
OSS/COS	共享存储	按量付费，¥0.1/GB·月级别，远低于 BE 本地 SSD
Flink (Paimon CDC)	实时写入与 Compaction	最大增量成本，TaskManager 数随写入量增长
StarRocks CN	纯计算	不带盘，分析查询少了可缩容，按需弹性
MySQL	OLTP 持久化	小规格即够用 (2C4G)，几百元/月
Redis	热缓存	小规格 (2-4GB)，几百元/月

关键判断：组件从 1 个变成 5 个，基础设施月费短期可能不降反升。真正的成本收益不在绝对值，在边际曲线——业务增长时，存储按 GB 线性付费、计算按核弹性扩缩，不再被「扩一个 BE 就绑一块盘」锁死。Flink 集群规模是最大变量，写入流量不大的情况下整体成本可控，写入量上去后需单独评估。

边际成本对比

场景	当前 (存算一体)	目标 (存算分离)
分析查询增加	扩 BE 节点，磁盘一起买	扩 CN 节点，只加计算
存储容量不够	扩 BE 节点，算力一起买	扩 OSS 容量，按量付费
后端点查压力增大	扩 BE 或加集群，I/O 竞争仍在	扩 MySQL 读副本/Redis 规格，成本可控
闲时降本	缩不下来 (BE 带着数据)	CN 可以缩，OSS 和 MySQL 保底

小蚕业务适配性

命中的点

外卖业务的读写模式天然分层——下单、改状态、查订单详情是 OLTP (高频点查)，看板、报表、用户分析是 OLAP (大扫聚合)。把两种负载塞进一个列存引擎里，就是目前 3 秒超时的根因。读写分离、分析和服务分层这个大方向，对小蚕是对的。

要不要上 Paimon ? 分情况看

Paimon 解决的核心问题是实时湖格式 (ACID、Upsert、Time Travel) 和共享存储多引擎访问。但它引入了一条额外的 CDC 链路和 Flink 运维负担。小蚕目前的核心痛点不是湖格式，是后端点查超时。

值得上 Paimon 的情况：

- 数据量已到几十 TB 且持续涨，存储成本成为主要矛盾
- 有多引擎共享同一份数据的需求 (StarRocks + Flink + Spark 都要读)
- 需要 Time Travel 做数据回溯 (财务对账、历史快照查询)

· 有专门的数据平台团队维护 Flink 和 Paimon 集群

如果以上不满足，更简方案：应用层双写——写数据时同时写 StarRocks 和 MySQL，StarRocks 专注分析，MySQL 扛后端点查，Redis 做热缓存。同样解决 3 秒超时，组件更少、运维更轻、往后再按需补 Paimon。这个路径可以分步走——先做 MySQL/Redis 服务层，跑通了再看要不要加 Paimon 做湖格式，不用一步到位。

综合判断：存算分离、读写分层的方向没错，但不必一步到位到 Paimon。核心矛盾是 OLTP 不该打 OLAP 数据库——先把